02

MGU BCA - 3RD SEM

# Python Programming

Module 2 - Functions and Data Structures

**By Aben Sojan**

Exam - 16 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

**EXAM PATTERN (from real MGU question papers)**

Exam: MG3SECBCA200 Python Programming - 50 marks, 1 hour 30 minutes (MGU Honours model paper, mgu.ac.in, Oct 2025).

Part A: 10 x 2 = 20 marks, answer ALL - short questions from every module.

Part B: answer any 3 of 5 x 5 = 15 marks - programs and detailed concept answers.

Part C: answer any 1 of 2 x 15 = 15 marks - one long essay or program.

Module 2 in the model paper: dict key-value, import forms, Numpy array (Part A); assert program, dictionary methods (Part B); list methods essay (Part C).

TOPIC 1

# Built-in Functions

1

Built-in functions are ready-made and need no import.

print(), len(), type(), input(), range() are used almost daily.

max(), min(), sum(), sorted(), abs() work on numbers and collections.

Example: len("hello") is 5; max(3, 9, 4) is 9; sum([1, 2, 3]) is 6.

QUESTION BANK

**1 What are built-in functions? Explain any five with examples.**

IMPORTANT

Built-in functions are ready-made functions that come with Python, so we can use them directly without importing anything or writing them ourselves.

They handle the most common jobs in every program, like showing output, checking data types, and working with numbers and collections.

print() shows output on the screen, for example print('Hello') displays Hello.

len() counts the items in a collection, for example len('hello') gives 5 because there are five letters.

type() tells the data type of a value, for example type(10) gives <class 'int'>.

range() creates a sequence of numbers, for example list(range(5)) gives [0, 1, 2, 3, 4], and it is mainly used with for loops.

max() and min() pick the largest and smallest value, for example max(3, 9, 4) gives 9 and min(3, 9, 4) gives 3.

sum() adds up all numbers in a collection, for example sum([1, 2, 3]) gives 6.

Example code:

```
print(len('hello'))
print(max(3, 9, 4))
print(sum([1, 2, 3]))
```

The output is 5, 9 and 6.

Because these functions are always available, they make programs shorter and save the programmer from rewriting common logic.

## 2 Explain range() with its three forms.

IMPORTANT

range() is a built-in function that creates a sequence of numbers, and it is most often used to control how many times a for loop runs.

It never includes the stop value; it always stops one number before it.

First form, range(stop), starts from 0 and goes up to stop minus 1, for example range(5) gives 0 1 2 3 4.

Second form, range(start, stop), starts from start and goes up to stop minus 1, for example range(2, 6) gives 2 3 4 5.

Third form, range(start, stop, step), jumps by step each time, for example range(1, 10, 2) gives 1 3 5 7 9.

The step can also be negative, which counts backwards, for example range(5, 0, -1) gives 5 4 3 2 1.

range() does not make a list directly; it makes a range object that produces numbers one by one, which saves memory.

Example code:

```
for i in range(3):
print(i)
```

This prints 0, 1 and 2 on separate lines.

So range() is the standard way to repeat an action a fixed number of times in Python.

**3** **Write a user-defined function to find the largest of two numbers.**

IMPORTANT

Define with def, compare with if-else and return the bigger value.

```
def largest(a, b):
if a > b:
return a
else:
return b
print(largest(10, 25))   # 25
```

Calling largest(10, 25) runs the function body with a = 10 and b = 25.

The function returns the greater of its two arguments.

TOPIC 2

## Function Definition, Call, Scope and Default Parameters

2

def name(parameters): defines a function; calling name(values) runs it.

return sends a result back to the caller; without return the function gives None.

Local variables exist only inside the function; global variables are visible everywhere.

Lifetime: a local variable is born when the function is called and dies when it ends.

Default parameters give a fallback value: def greet(name="Guest").

QUESTION BANK

**4** **Write a Python function to calculate and return the factorial of a given number. (MG2DSCCSC100, Mar 2024)**

REPEATED

Loop from 1 to n multiplying into a running total, then return it.

```
def factorial(n):
fact = 1
for i in range(1, n + 1):
fact *= i
return fact
print(factorial(5))   # 120
```

fact starts at 1 because multiplying by 0 would ruin the result; return sends the answer back.

Recursive version: return 1 if n <= 1 else n * factorial(n - 1).

The function builds n! step by step and returns the final product.

**5 What is recursion? Explain how a recursive function works with a program. (Nov 2025, Mar 2024)**

REPEATED

Recursion is when a function calls itself to solve a smaller version of the same problem.

Every recursive function needs a base case that stops the calling, otherwise it never ends.

Example: factorial(3) = 3 * factorial(2) = 3 * 2 * factorial(1) = 3 * 2 * 1 = 6.

```
def factorial(n):
if n <= 1:                  # base case
return 1
return n * factorial(n - 1) # recursive call
print(factorial(5))             # 120
```

Each call waits for the smaller call to finish, then multiplies and returns up the chain.

Base case plus a self-call on a smaller input is the whole recursion pattern.

**6 How do you define and call a function? Explain with an example.**

IMPORTANT

A function is a named block of code that does one job, and it lets us reuse the same logic many times without rewriting it.

A function is defined with the def keyword, followed by the function name, brackets with parameters inside, a colon, and then the indented body.

To use the function we call it by its name with actual values in brackets; this is called the function call.

The values passed at the call are called arguments, and the names written in the definition are called parameters.

Example code:

```
def add(a, b):
return a + b


print(add(4, 5))
```

Here add is defined with parameters a and b; calling add(4, 5) runs the body with a = 4 and b = 5.

The return statement sends the result 9 back to the caller, so print shows 9 as the output.

Without a call, the function body never runs; the definition only stores the code.

Functions keep the program organized, because each part of the work has its own name and place.

## 7 Explain local and global scope with an example.

IMPORTANT

The scope of a variable is the part of the program where that variable can be seen and used.

Local scope means the variable is created inside a function, so it works only inside that function and is destroyed when the function ends.

Global scope means the variable is created outside all functions, at the top level, so every function in the program can read it.

A function cannot change a global variable by a plain assignment; assignment inside a function creates a new local variable instead.

To change a global variable inside a function we must declare it with the global keyword.

Example code:

```
x = 10
def show():
y = 5
print(x + y)
show()
```

Here x is global, so the function can read it; y is local to show() and exists only while show() runs.

The output is 15.

Trying to use y outside the function would give a NameError, which proves that local variables stay hidden.

So scope decides the lifetime and visibility of every variable in the program.

## 8 What are default parameters? Give an example.

IMPORTANT

A default parameter is a parameter that already has a value fixed in the function definition itself.

If the caller passes a value for it, that value is used; if the caller skips it, the default value is used automatically.

Default parameters make functions flexible, because the common case works without any argument while special cases can still pass their own value.

In the definition, default parameters must come after all the normal parameters.

A common rule is to avoid using a changeable object like a list as a default value, because the same object is shared across all calls.

Example code:

```
def greet(name='Guest'):
print('Hello', name)


greet()
greet('Aben')
```

The first call skips the argument, so the default 'Guest' is used and it prints Hello Guest.

The second call passes 'Aben', which replaces the default, so it prints Hello Aben.

Thus default parameters save the caller from passing values that are the same in most calls.

## 9 What does return do? What happens without it?

IMPORTANT

The return statement ends a function immediately and hands a value back to the caller.

After return runs, no further lines of the function body are executed.

A function can return any value: a number, a string, a list, or even another function.

A function can have many return statements, but only the first one reached actually runs.

If a function has no return statement, or just a bare return, Python automatically returns the special value None.

That means a function without return still does its work, like printing, but it gives nothing back to the caller.

Example code:

```
def add(a, b):
return a + b
```

```
print(add(2, 3))
```

Here return sends 5 back, so the output is 5; if the return line were removed, the output would be None.

So return is the way a function communicates its result to the rest of the program.

**10** **Which keywords are used in functions? Explain with examples.**

IMPORTANT

def creates a function: def add(a, b): defines add.

return sends a value back to the caller and ends the function.

lambda builds a small anonymous function: f = lambda x: x * 2.

global lets a function modify a variable defined outside it.

nonlocal lets a nested function modify its enclosing function's variable.

yield turns a function into a generator that produces values one by one.

Example:

```
def square(x):
return x * x
print(square(6))   # 36
```

def and return are the core pair; lambda, global, nonlocal and yield handle special cases.

**11** **Write a Python program which defines three functions to calculate the area of a rectangle, square and circle based on the user choice.**

IMPORTANT

Three small functions plus a menu that calls the chosen one.

```
import math
def area_rect(l, b):
return l * b
def area_square(s):
return s * s
def area_circle(r):
return math.pi * r * r
print("1. Rectangle  2. Square  3. Circle")
ch = int(input("Choice: "))
if ch == 1:
l = float(input("Length: "))
```

```
b = float(input("Breadth: "))
print("Area:", area_rect(l, b))
elif ch == 2:
s = float(input("Side: "))
print("Area:", area_square(s))
elif ch == 3:
r = float(input("Radius: "))
print("Area:", area_circle(r))
else:
print("Invalid choice")
```

Each function holds one area formula; the if-elif menu routes to the right one.

Separate functions keep each formula clean and reusable.

TOPIC 3

## Command Line Arguments, Lambda and Assert

3

Command line arguments are values typed after the program name; read them from sys.argv.

sys.argv[0] is the program name itself; sys.argv[1] is the first real argument.

lambda makes a one-line anonymous function: square = lambda x: x * x.

assert tests a condition; if it is False the program stops with AssertionError.

Example: assert age > 0, "age must be positive".

QUESTION BANK

### 12 What are command line arguments? Explain with sys.argv.

IMPORTANT

Command line arguments are values given to a program when it is started from the terminal, so the program can change its behaviour without editing the code.

They are typed after the program name, for example python prog.py 10 20 makes 10 and 20 the arguments.

Python collects these values in the list sys.argv from the sys module.

sys.argv[0] is always the name of the program file itself, and the real arguments start from sys.argv[1].

All values in sys.argv are strings, so we must convert them with int() or float() before doing maths.

Example code:

```
import sys
print('Program:', sys.argv[0])
print('First arg:', sys.argv[1])
```

Running python prog.py 10 20 prints Program: prog.py and First arg: 10.

If we need the number 10 for addition, we write int(sys.argv[1]).

Command line arguments are useful for scripts, tools and automation, where the same program must handle different inputs each time.

## 13 What is a lambda function? Give two examples.

IMPORTANT

A lambda function is a small unnamed function written in a single line, used when we need a short function only once.

Its syntax is lambda arguments: expression, where the expression is calculated and its result is automatically returned.

Unlike a normal function, a lambda body can contain only one expression; it cannot have statements or multiple lines.

The result of lambda is a function object, so we usually store it in a variable and then call it.

Example code:

```
square = lambda x: x * x
add = lambda a, b: a + b
print(square(6), add(2, 3))
```

The output is 36 5, because square(6) gives 36 and add(2, 3) gives 5.

Lambdas are most useful where a short function is passed to another function, for example key = lambda s: len(s) inside sorted() to sort strings by length.

They are also commonly used with map() and filter() to process lists without writing a full def function.

## 14 Explain the assert statement with an example.

IMPORTANT

The assert statement is a debugging tool that checks whether a condition is true while the program runs.

Its form is assert condition, message, where the message is optional.

If the condition is True, the program continues normally as if nothing happened.

If the condition is False, Python raises an AssertionError with the message and stops the program at that point.

assert is meant to catch the programmer's own mistakes during development and testing, not to handle normal user errors.

Example code:

```
age = int(input('Enter age: '))
assert age > 0, 'age must be positive'
print('Age accepted')
```

If the user types 25, the assertion passes and Age accepted is printed.

If the user types -5, the program stops with AssertionError: age must be positive.

So assert acts like a guard that fails loudly the moment the program reaches an impossible state.

## 15 What is the use of the sys module in Python?

IMPORTANT

The sys module gives access to interpreter-level information and functions.

sys.argv holds command line arguments: sys.argv[0] is the script name and the rest are user inputs.

sys.exit() stops the program immediately, which is useful on fatal errors.

sys.version shows the Python version; sys.path lists where Python searches for modules.

Example:

```
import sys
print(sys.argv)   # ['prog.py', '10'] when run as python3 prog.py 10
```

sys is the bridge between a running program and the interpreter around it.

TOPIC 4

4

# Importing User Defined Modules

A module is a .py file holding functions, classes and variables for reuse.

import mymath loads your own file mymath.py so you can call mymath.add(2, 3).

from mymath import add imports only one function; then call add(2, 3) directly.

import numpy as np gives the module a short nickname for easy use.

QUESTION BANK

16 **What is a module? How do you import a user defined module?**

IMPORTANT

A module is simply a Python file that contains functions, variables or classes which we want to reuse in other programs.

Modules let us write useful code once and then use it in many programs, instead of copying the same code everywhere.

To make a user defined module, we save our functions in a file, for example mymath.py containing def add(a, b): return a + b.

In the program that needs it, we write import mymath, and Python loads the file.

Then we call its functions with the module name as a prefix, for example mymath.add(4, 5) gives 9.

The module file must be in the same folder as the program, or in a folder Python searches, for the import to work.

Example code:

```
import mymath
print(mymath.add(4, 5))
```

The output is 9.

So modules are the basic building blocks for organizing large programs into clean, reusable parts.

### 17 Distinguish between import math and from math import sqrt.

IMPORTANT

Both statements bring the math module into the program, but they differ in what names they expose and how the functions are called.

import math loads the whole module, and every function must be called with the module name as prefix, for example math.sqrt(16) gives 4.0.

This form keeps the namespace clean, because it is always clear where sqrt comes from.

from math import sqrt loads only the sqrt function into the current namespace, so we call it directly as sqrt(16) without the math prefix.

This form is shorter to write, which is handy when a function is used many times.

The risk of the from form is a name clash: if our own program already has a name sqrt, the imported one silently replaces it.

Full import math is safer in big programs where names may clash; from-import is fine in small scripts for brevity.

In short, import math is the safer long form, while from math import sqrt is the shorter direct form.

### 18 Define a package in Python.

IMPORTANT

A package is a folder of related Python modules grouped under one name for easy import.

The folder contains an __init__.py file (which may be empty) marking it as a package.

Example layout: mypack/__init__.py, mypack/maths.py, mypack/text.py.

It is imported with import mypack.maths or from mypack import maths.

Packages keep big projects organized instead of dumping every module in one folder.

A package is a directory of modules plus __init__.py, imported by its dotted name.

TOPIC 5

## Lists, Tuples and Dictionaries

5

Lists are ordered and changeable: [1, 2, 3]; tuples are ordered but fixed: (1, 2, 3).

Dictionaries store key-value pairs: {"name": "Aben", "age": 20}.

Common list methods: append(), extend(), insert(), remove(), pop(), sort(), reverse().

Mutable objects can change after creation: list, dict, set.

Immutable objects cannot change: tuple, string, int, float.

Passing a list to a function lets the function change the original list.

QUESTION BANK

**19** **Differentiate between pop(), popitem() and clear() in dictionaries with suitable examples. (Model Paper, MG2DSCCSC100, Nov 2025)**

SURE MARK

pop(key) removes the entry with the given key and returns its value; it raises KeyError if the key is missing (unless a default is given).

popitem() removes and returns the last inserted key-value pair as a tuple.

clear() removes all entries, leaving an empty dictionary.

Example:

```
d = {"a": 1, "b": 2, "c": 3}
print(d.pop("b"))      # 2, d is now {"a": 1, "c": 3}
print(d.popitem())     # ('c', 3), last pair removed
d.clear()
print(d)               # {}
```

Use pop when you know the key and want its value; popitem for last-in cleanup; clear to reset the whole dict.

In short: pop removes by key, popitem removes the last pair, clear empties everything.

**20** **Distinguish between lists, tuples and dictionaries.**

IMPORTANT

A list is an ordered collection that can be changed after creation, written in square brackets, for example [1, 2, 3].

A tuple is an ordered collection that cannot be changed after creation, written in round brackets, for example (1, 2, 3).

A dictionary is an unordered collection of key-value pairs, written in curly braces, for example {'name': 'Aben'}.

Lists support changes like l[0] = 9, while doing t[0] = 9 on a tuple raises a TypeError.

Dictionary items are fetched by key instead of position, for example d['name'] gives 'Aben'.

Lists are faster to modify; tuples are faster to access and can be used as dictionary keys because they are unchangeable.

Example code:

```
l = [1, 2, 3]
```

```
l[0] = 9
d = {'name': 'Aben'}
print(l, d['name'])
```

The output is [9, 2, 3] Aben.

As a rule, use lists for collections that change, tuples for fixed data, and dictionaries for labelled data.

## 21 Explain any five list methods with examples.

IMPORTANT

List methods are built-in functions of the list object that add, remove or rearrange its items.

append(x) adds x at the end of the list, for example [1, 2].append(3) makes it [1, 2, 3].

insert(i, x) puts x at position i and shifts later items right, for example [1, 3].insert(1, 2) makes it [1, 2, 3].

remove(x) deletes the first occurrence of x, for example [1, 2, 2].remove(2) leaves [1, 2].

pop() removes the last item and returns it, for example [1, 2, 3].pop() gives 3 and the list becomes [1, 2]; pop(i) removes the item at position i.

sort() arranges the items in ascending order, for example [3, 1, 2].sort() makes it [1, 2, 3].

reverse() flips the order of the items, for example [1, 2, 3].reverse() makes it [3, 2, 1].

All these methods except pop change the list in place and return None, so we must not assign their result back to the list.

Mastering these five methods covers almost all everyday list work in Python.

## 22 What are mutable and immutable objects? Give examples.

IMPORTANT

Mutable objects are objects that can be changed after they are created, without making a new object.

The main mutable types are list, dictionary and set, for example we can add or remove items from them freely.

Immutable objects are objects that can never be changed after creation; any operation that looks like a change actually makes a new object.

The main immutable types are tuple, string, int and float.

Example code:

```
l = [1, 2]
l.append(3)
```

```
t = (1, 2)
```

After this, l becomes [1, 2, 3] because lists are mutable, but there is no way to change t.

Trying t[0] = 9 raises a TypeError, which proves tuples are immutable.

This difference matters when objects are passed to functions: mutable objects can be changed by the function, while immutable ones stay safe.

## 23 Explain how lists behave when passed to a function.

IMPORTANT

When a list is passed to a function, Python passes the reference to the same list object, not a copy of it.

So the parameter inside the function and the original list outside both point to the very same object in memory.

Any change the function makes through the parameter, like lst[i] = x, directly changes the caller's original list.

This is different from immutable values like numbers, where the function only gets a copy of the value.

Example code:

```
def double(lst):
for i in range(len(lst)):
lst[i] = lst[i] * 2


nums = [1, 2, 3]
double(nums)
print(nums)
```

The output is [2, 4, 6], which proves the original list nums was changed by the function.

This behaviour is powerful but needs care: if we want the function to leave the original untouched, we must pass a copy like nums[:].

## 24 Show with an example how to add a key-value pair to a dictionary.

IMPORTANT

Assigning to a new key inserts the pair; assigning to an existing key updates it.

```
d = {"name": "Asha"}
d["age"] = 20          # new key: added
d["name"] = "Asha R"   # existing key: updated
print(d)               # {'name': 'Asha R', 'age': 20}
```

The update() method can add several pairs at once: d.update({"city": "Kochi"}).

d[new_key] = value is the standard insert-or-update step.

## 25 Discuss the difference between lists and tuples with examples.

IMPORTANT

Lists are mutable: items can be changed, added or removed after creation; they are written in square brackets.

Tuples are immutable: once created they cannot change; they are written in round brackets.

Example:

```
lst = [1, 2, 3]
lst[0] = 9        # allowed
tpl = (1, 2, 3)
# tpl[0] = 9      # TypeError: tuples cannot change
```

Lists suit collections that grow or change (marks, cart items); tuples suit fixed records (coordinates, dates).

Tuples are slightly faster and can be dictionary keys; lists cannot.

Mutability is the core difference, and it decides which one to use.

## 26 What is an empty dictionary? Give an example program.

IMPORTANT

An empty dictionary has no key-value pairs; it is the starting point you fill later.

It is created with {} or dict().

```
d = {}
print(d)            # {}
print(len(d))       # 0
d["id"] = 101       # now it holds one pair
print(d)            # {'id': 101}
```

Programs often start empty and add entries inside a loop, for example when counting word frequencies.

{} or dict() makes the empty container; assignments fill it.

## 27 How do you concatenate two lists? Give an example.

IMPORTANT

The + operator joins two lists into a new one; extend() appends all items of one list into another.

```
a = [1, 2]
b = [3, 4]
c = a + b
print(c)            # [1, 2, 3, 4]
a.extend(b)
print(a)            # [1, 2, 3, 4]
```

+ leaves both originals unchanged and makes a new list; extend() changes the first list in place.

Use + for a new combined list and extend() to grow an existing one.

## 28 What is indexing and negative indexing in tuples? Explain with examples.

IMPORTANT

Indexing reads tuple items by position starting at 0: t[0] is the first item.

Negative indexing counts from the end: t[-1] is the last item and t[-2] the one before it.

```
t = (10, 20, 30, 40)
print(t[1])     # 20
print(t[-1])    # 40
print(t[1:3])   # (20, 30), slicing works too
```

Out-of-range indexes raise IndexError, exactly like lists.

Zero-based forward indexes plus minus-one-based backward indexes cover every position.

**29** **Write a Python program to sort a group of strings into alphabetical order.**

IMPORTANT

The sorted() function returns a new sorted list; the sort() method sorts a list in place.

```
words = ["mango", "apple", "cherry"]
words.sort()
print(words)   # ['apple', 'cherry', 'mango']
```

Sorting is alphabetical (technically by character code); sorted(words) would keep the original untouched.

For case-insensitive order use sort(key=str.lower).

sort() or sorted() arranges strings A to Z in one call.

TOPIC 6

## Math Module and Numpy

6

import math gives sqrt(), pow(), factorial(), ceil(), floor() and the constant pi.

Numpy handles fast numeric arrays: import numpy as np.

np.array([1, 2, 3]) makes an array; maths applies to every element at once.

np.arange(10), np.zeros(3), np.ones((2, 2)) create ready-made arrays.

Numpy is far faster than Python lists for large numeric calculations.

QUESTION BANK

**30** **Explain the math module with any five functions.**

IMPORTANT

The math module is Python's built-in library of mathematical functions, loaded with import math before use.

It provides exact, ready-made maths so we do not have to write formulas ourselves.

math.sqrt(x) gives the square root, for example math.sqrt(16) gives 4.0.

math.pow(x, y) gives x raised to the power y, for example math.pow(2, 3) gives 8.0.

math.factorial(n) gives the factorial of a whole number, for example math.factorial(5) gives 120.

math.ceil(x) rounds up to the nearest whole number, for example math.ceil(4.2) gives 5, while math.floor(x) rounds down, for example math.floor(4.8) gives 4.

The module also gives constants like math.pi, which is 3.14159..., used in circle calculations.

Example code:

```
import math
```

```
print(math.sqrt(16), math.factorial(5), math.ceil(4.2))
```

The output is 4.0 120 5.

So the math module covers the everyday maths needs of most programs in one import.

## 31 What is Numpy? How is it better than lists for numbers?

IMPORTANT

Numpy is a popular Python library for fast numerical work, built around a special object called the array.

A Numpy array holds many numbers of the same type in one compact block, unlike a Python list which can mix types.

Python list loops run in slow Python code, while Numpy's operations run in fast compiled C code, so large calculations finish much quicker.

Arrays use less memory than lists because the numbers are stored tightly together without extra object overhead.

Numpy supports whole-array maths, so arr * 2 doubles every element at once with no loop to write.

It also gives ready functions like np.sum, np.mean, np.max and np.min for instant statistics.

Example code:

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr * 2)
```

The output is [2 4 6], showing the whole array doubled in one step.

Because of this speed and simplicity, Numpy is the base of data science libraries like Pandas and Scikit-learn.

## 32 Write code to create a Numpy array and find its sum and mean.

IMPORTANT

A Numpy array is created with np.array(), which turns a Python list into a fast Numpy array.

Once the array exists, np.sum() adds all its elements and np.mean() gives their average.

The convention is to import Numpy as np with import numpy as np before doing anything else.

Example code:

```
import numpy as np
arr = np.array([10, 20, 30, 40])
print('Sum:', np.sum(arr))
```

```
print('Mean:', np.mean(arr))
```

Here arr holds the four numbers 10, 20, 30 and 40.

np.sum(arr) adds them to 100, and np.mean(arr) divides by 4 to give 25.0.

The output is Sum: 100 and Mean: 25.0.

This pattern of creating an array and calling one-line statistics functions is the core of everyday Numpy use.

33 **Write Python code using Numpy to generate an array of even numbers between 10 and 20.**

IMPORTANT

numpy.arange(start, stop, step) builds evenly spaced values; a step of 2 gives even numbers.

```
import numpy as np
arr = np.arange(10, 21, 2)
print(arr)   # [10 12 14 16 18 20]
```

arange(10, 21, 2) starts at 10, stops before 21 and steps by 2.

arange with step 2 is the direct Numpy way to list even numbers.